《嵌入式系统》实验报告 2

学号：21013025 姓名：单楚雄 班级：计 211 日期：2024.5.16

成绩：_____ 指导教师：罗飞

实验名称：嵌入式 Linux 开发环境	实验地点：
实验仪器：Linux 实验环境	

一、实验目的：

1、 熟悉命令行操作方式进行用户和用户组管理
2、 掌握 vi 编辑器的进入与退出方法
3、 了解文本编辑器的三种模式
4、 熟练掌握使用 vi 编辑器进行编辑、选择及操作文本文件的命令
5、 安装和交叉编译环境
6、 理解和掌握 bootloader 的使用方法

二、实验内容：

1. 创建用户（以 zhangsan 为例）

(1) 创建一个新用户 zhangsan
(2) 查看/etc/passwd 文件的最后一行内容，并记录
(3)查看/etc/shadow 文件的最后一行内容，并记录
(4) 给新建的用户设置密码
(5) 查看/etc/shadow 文件的最后一行内容，记录并说明变化
(6) 使用 zhangsan 用户登录系统，测试能否登录成功

简述实验的详细步骤

（1），sudo adduser zhangsan，输入密码 ecust。

（2），tail -n 1 /etc/passwd

```
ecust@Ubuntu:~$ tail -n 1 /etc/passwd
zhangsan:x:1002:1002::/home/zhangsan:
```

（3）sudo tail -n 1 /etc/shadow

```
ecust@Ubuntu:~$ sudo tail -n 1 /etc/shadow
zhangsan:$6$eUoTXp.C$ER8Zs24gyP9sYFv70a8UaUDPZ6OI3WKTWBpUuR/UYTm0nFMd2//C4D37nwd
2DXlz5vNHgq0fTCwv3u8x6Hyfi.:19866:0:99999:7:::
```

（4）sudo passwd zhangsan

```
ecust@Ubuntu:~$ sudo passwd zhangsan
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
```

（5） sudo tail -n 1 /etc/shadow

```
ecust@Ubuntu:~$ sudo tail -n 1 /etc/shadow
zhangsan:$6$eUoTXp.C$ER8Zs24gyP9sYFv70a8UaUDPZ6OI3WKTWBpUuR/UYTm0nFMd2//C4D37nwd
2DXlz5vNHgq0fTCwv3u8x6Hyfi.:19866:0:99999:7:::
```

变化：密码字段现在包含了加密后的密码哈希值。

（6）su - zhangsan

```
ecust@Ubuntu:~$ su zhangsan
Password:
zhangsan@Ubuntu:/home/ecust$
```

2. 创建用户 2

（1）使用 1 的步骤创建新用户 user

（2）更改 zhangsan 所属群组为 root

（3）查看/etc/passwd 文件，记录 zhangsan 用户和 user 用户的属组情况

（4）删除用户 user

简述实验的详细步骤。

1，sudo adduser user

```
ecust@Ubuntu:~$ sudo adduser user
[sudo] password for ecust:
Adding user `user' ...
Adding new group `user' (1003) ...
Adding new user `user' (1003) with group `user' ...
Creating home directory `/home/user' ...
Copying files from `/etc/skel' ...
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
Changing the user information for user
Enter the new value, or press ENTER for the default
        Full Name []:
        Room Number []:
        Work Phone []:
        Home Phone []:
        Other []:
Is the information correct? [Y/n] Y
```

2，sudo usermod -aG root zhangsan

3，grep -E "zhangsan|user" /etc/passwd

（4），sudo deluser user

```
ecust@Ubuntu:~$ sudo deluser user
Removing user `user' ...
Warning: group `user' has no more members.
Done.
```

3. 组的管理

（1）创建一个新组，组名为 stuff

（2）查看/etc/group 文件的最后一行内容，并记录

（3）创建一个新账户 test，并将其起始组合附属组都设置为 stuff

（4）查看/etc/group 文件中的最后一行内容，记录并说明变化

（5）设置 stuff 组密码

（6）在 stuff 组中删除用户 test

（7）查看/etc/group 文件中的最后一行，记录并说明变化

简述实验的详细步骤。

1，sudo groupadd stuff

2，tail -n 1 /etc/group

```
ecust@Ubuntu:~$ tail -n 1 /etc/group
stuff:x:1003:
```

3，sudo useradd -g stuff -G stuff test

sudo passwd test

```
ecust@Ubuntu:~$ sudo useradd -g stuff -Gstuff test
ecust@Ubuntu:~$ sudo passwd test
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
```

4，tail -n 1 /etc/group

```
ecust@Ubuntu:~$ tail -n 1 /etc/group
stuff:x:1003:test
```

5，sudo gpasswd stuff

```
ecust@Ubuntu:~$ sudo gpasswd stuff
Changing the password for group stuff
New Password:
Re-enter new password:
```

6，sudo gpasswd -d test stuff

7，tail -n 1 /etc/group

```
ecust@Ubuntu:~$ sudo gpasswd -d test stuff
Removing user test from group stuff
ecust@Ubuntu:~$ tail -n 1 /etc/group
stuff:x:1003:
```

4. vim 编辑器的使用

（1）进入 vim 编辑器，建立一个文件，如 file.c；进入插入方式，输入一个 C 语言程序的各行内容，故意制造几处错误。最后，将该文件存盘。回到 shell 状态下。

（2）运行 gcc file.c -o myfile，编译该文件，会发现错误提示。说明其含义。

（3）重新进入 vim 编辑器，对该文件进行修改。然后存盘，退出 vi 编辑器。重新编译该文件，如

果编译通过了，可以利用./myfile 运行该程序。

（4）运行 man date > file10，然后 vi file10。使用 x，dd 等命令删除某些文本行。使用 u 命令复原此前的情况。使用 c、r、s 等命令修改文本内容。使用检索命令进行给定模式的检索。说明以上各个命令的作用。

简述实验的详细步骤。

1，vim file.c

输入以下内容：

```
#include <stdio.h>
printf("Hello,World!\n")
return 0;
}
```

2，gcc file.c -o myfile

```
ecust@Ubuntu:~$ gcc file.c -o myfile
file.c: In function 'main':
file.c:4:2: error: expected ';' before 'return'
  return 0;
  ^
ecust@Ubuntu:~$
```

该错误提示表示在第 4 行的‘return’之前缺少分号。

3，

```
ecust@Ubuntu:~$ gcc file.c -o myfile
ecust@Ubuntu:~$ ./myfile
Hello,World!
ecust@Ubuntu:~$
```

4，man date > file10

vi file10

x：删除光标所在的字符。

dd：删除光标所在的整行。

u：撤销上一个操作。

cw：修改光标处的单词。

r：替换光标所在的字符。

s：替换光标所在的字符并进入插入模式。

/pattern：搜索给定的模式。

进一步完成实验指导手册中的实验，并完成如下内容：

5. 简述安装 ToolChain 的主要步骤（说明 source 命令的作用）。

source 命令的作用是在当前 shell 环境中执行指定脚本文件，并将其中的变量和函数应用到当前的 shell 环境中。

1.准备环境，并安装 ToolChain

建立工作目录：

ecust@...:~$mkdir/home/ecust/workplace

ecust@...:~$cd/home/ecust/workplace

2.ToolChain 安装：

ecust@...:~/workplace$ tar jxvf /home/ecust/samba_share/embed/ToolChain/toolchain-4.5.1-farsight.tar.bz2

ecust@...:~/workplace$ln-stoolchain-4.5.1-farsighttoolchain

ecust@...:~/workplace$vim~/.bashrc

在文件的末尾输入：

exportPATH=/home/ecust/workplace/toolchain/bin:$PATH

输入 arm，按下 Tab 键

```
ecust@Ubuntu:~/workplace$ arm
arm2hpdl                        arm-none-linux-gnueabi-addr2line
arm-linux-addr2line             arm-none-linux-gnueabi-ar
arm-linux-ar                    arm-none-linux-gnueabi-as
arm-linux-as                    arm-none-linux-gnueabi-c++
arm-linux-c++                   arm-none-linux-gnueabi-cc
arm-linux-cc                    arm-none-linux-gnueabi-c++filt
arm-linux-c++filt               arm-none-linux-gnueabi-cpp
arm-linux-cpp                   arm-none-linux-gnueabi-g++
arm-linux-g++                   arm-none-linux-gnueabi-gcc
arm-linux-gcc                   arm-none-linux-gnueabi-gcc-4.5.1
arm-linux-gcc-4.5.1             arm-none-linux-gnueabi-gccbug
arm-linux-gccbug                arm-none-linux-gnueabi-gcov
arm-linux-gcov                  arm-none-linux-gnueabi-gprof
arm-linux-gprof                 arm-none-linux-gnueabi-ld
arm-linux-ld                    arm-none-linux-gnueabi-ldd
arm-linux-ldd                   arm-none-linux-gnueabi-nm
arm-linux-nm                    arm-none-linux-gnueabi-objcopy
arm-linux-objcopy               arm-none-linux-gnueabi-objdump
arm-linux-objdump               arm-none-linux-gnueabi-populate
arm-linux-populate              arm-none-linux-gnueabi-ranlib
arm-linux-ranlib                arm-none-linux-gnueabi-readelf
arm-linux-readelf               arm-none-linux-gnueabi-size
arm-linux-size                  arm-none-linux-gnueabi-strings
arm-linux-strings               arm-none-linux-gnueabi-strip
arm-linux-strip
```

成功配置环境。ToolChain 安装成功。

6．编辑并编译 C 语言 hello.c，该执行文件可输出：“Hello, world!”；分别写出基于 X86、ARM 架构的编辑、编译步骤，并用 file 查看 X86、ARM 架构下可执行文件的格式，说明其区别。

基于 x86 架构的编辑和编译步骤：

1. 编辑 hello.c 文件：

```
#include <stdio.h>

int main() {
    printf("Hello, world!\n");
    return 0;
}
```

2，编译 hello.c 文件

gcc hello.c -o hello_x86

3，查看可执行文件格式：

file hello_x86

```
ecust@Ubuntu:~$ gcc hello.c -o hello_x86
ecust@Ubuntu:~$ file hello_x86
hello_x86: ELF 64-bit LSB  executable, x86-64, version 1 (SYSV), dynamically lin
ked (uses shared libs), for GNU/Linux 2.6.24, BuildID[sha1]=655b36c3ebe6ae56a019
049171c5500f6d5ee127, not stripped
```

基于 ARM 架构的编辑和编译步骤：

1. 编辑 hello.c 文件

2. 交叉编译 hello.c 文件

arm-linux-gnueabihf-gcc hello.c -o hello_arm

3.查看可执行文件格式：

file hello_arm

输出 hello_arm: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux-armhf.so.3

区别：

x86 架构：可执行文件格式为 ELF 64-bit LSB executable，适用于 x86_64（64 位）架构的处理器，使用的是 /lib64/ld-linux-x86-64.so.2 作为动态链接器。

ARM 架构：可执行文件格式为 ELF 32-bit LSB executable，适用于 ARM 架构的处理器，使用的是 /lib/ld-linux-armhf.so.3 作为动态链接器。

7．bootLoader 程序的编译

说明编译 Bootloader 程序的步骤：

进入工作目录：$cd/home/ecust/workplace

解压 uboot：

$tar-xvf/home/ecust/samba_share/embed/U-Boot/uboot-fs210_V5.3.tar

进入 uboot 目录：

$cduboot-fs210_V5.3/

修改交叉工具链的路径：

即修改 Makefile 中的 144 行：

ecust@Ubuntu:~/workplace/uboot-fs210_V5.3$vimMakefile

修改结果如下：

#CROSS_COMPILE=/usr/local/toolchain/toolchain-4.3.2/bin/arm-none-linux-gnueabi-

CROSS_COMPILE=/home/ecust/workplace/toolchain/bin/arm-none-linux-gnueabi-

执行命令编译 NandFlashU-Bootecust@Ubuntu:~/workplace/uboot-fs210_V5.3$makedistclean

该命令含义：

ecust@Ubuntu:~/workplace/uboot-fs210_V5.3$makefs210_nand_config

ecust@Ubuntu:~/workplace/uboot-fs210_V5.3$make

编译完成之后，会生成 bootloader 映像文件 u-boot.bin，利用 ls 查看该文件。

```
ecust@Ubuntu:~/workplace/uboot-fs210_V5.3$ ls
api                              COPYING    lib_arm       onenand_ipl    tools
api_examples                     cpu        libfdt        other          u-boot
arm_config.mk                    CREDITS    lib_generic   post           u-boot.bin
board                            disk       MAINTAINERS   README         u-boot.dis
CHANGELOG                        doc        Makefile      README.FS210   u-boot.map
CHANGELOG-before-U-Boot-1.1.5    drivers    mkconfig      rules.mk       u-boot.srec
Changelog_Samsung                examples   nand_spl      sd_fusing
common                           fs         net           System.map
config.mk                        include    onenand_bl1   tags
ecust@Ubuntu:~/workplace/uboot-fs210_V5.3$
```

查看 bootloader 源码，说明 include/configs/fs210_nand.h 和 include/configs/fs210_sd.h 文件的区别：

ecust@... $ diff include/configs/fs210_nand.h include/configs/fs210_sd.h

```
ecust@Ubuntu:~/workplace/uboot-fs210_V5.3$ diff include/configs/fs210_nand.h inc
lude/configs/fs210_sd.h
234c234
< #define CONFIG_BOOTARGS       "root=/dev/mtdblock3 rootfstype=yaffs2 init=/ini
t console=ttySAC0,115200"
---
> #define CONFIG_BOOTARGS       "root=/dev/mmcblk0p2 rootfstype=ext4 init=/init
console=ttySAC0,115200"
637,638c637,638
< #define CFG_FASTBOOT_NANDBSP
< //#define CFG_FASTBOOT_SDMMCBSP
---
> //#define CFG_FASTBOOT_NANDBSP
> #define CFG_FASTBOOT_SDMMCBSP
647c647
< #define CONFIG_BOOTCOMMAND    "nand read 20008000 500000 500000; bootm 2000800
0"
---
> #define CONFIG_BOOTCOMMAND    "nand read C0008000 600000 400000; nand read 30A
00000 B00000 180000; bootm C0008000 30A00000"
649c649,650
< #define CONFIG_BOOTCOMMAND    "movi read kernel 20008000; movi read rootfs 30B
00000 300000; bootm 20008000 30B00000"
---
> //#define CONFIG_BOOTCOMMAND  "movi read kernel 20008000; bootm 20008000"
> #define CONFIG_BOOTCOMMAND    "movi read kernel 20008000; movi read rootfs 30A
00000 300000; bootm 20008000 3A00000"
```

区别：

启动参数 CONFIG_BOOTARGS:

fs210_nand.h 使用 NAND 闪存 (/dev/mtdblock3) 和 yaffs2 文件系统。

fs210_sd.h 使用 SD 卡 (/dev/mmcblk0p2) 和 ext4 文件系统。

快速启动配置:

fs210_nand.h 定义了 CFG_FASTBOOT_NANDSBP，表示启用 NAND 闪存的快速启动。

fs210_sd.h 定义了 CFG_FASTBOOT_SDMMCBSP，表示启用 SD 卡的快速启动。

启动命令 CONFIG_BOOTCOMMAND:

fs210_nand.h 配置从 NAND 闪存读取数据到特定内存地址并执行。

fs210_sd.h 配置从 SD 卡读取数据到特定内存地址并执行。

8. 如何将编译好的 bootloader 下载到目标板中？请说明主要步骤。

通过 USB 使用 fastboot 下载

1.进入 Fastboot 模式：

将目标板设置为 Fastboot 模式并连接到主机。

2.下载并烧录 Bootloader：

在主机上执行 Fastboot 命令：

```
fastboot flash bootloader u-boot.bin
```